Architecture Overview & Blueprint **2**
Steps performed **6**
I create s3 bucket **6**
I create dynamodb 7
Created SQS queue **8**
Allowed S3 bucket to drop notifications into SQS **8**
Linked SQS to S3 bucket for create event notifications **9**
Create policy LambdaImageProcessorRole **10**
LambdaImageProcessorRole has sqs:ReceiveMessage, sqs:DeleteMessage and sqs:GetQueueAtttributes permissions **11**
LambdaImageProcessorRole has dynamodb:PutItem permission **11**
Created Lambda function with LambdaImageProcessorRole **12**
Added image-processing-queue as trigger for the Lambda function **15**
Added Lambda code **16**
Deployed Lambda function **17**
Uploaded an image to S3 bucket **18**

Estimated Duration: 1 to 1.5 Hours
Focus Areas: Serverless Architecture, Decoupling, Event-Driven Processing, Least Privilege IAM, NoSQL Integration

Architecture Overview & Blueprint

This lab expands your AWS Solutions Architect portfolio into **Event-Driven Serverless Architectures**. Instead of running dedicated compute infrastructure like EC2 instances behind a load balancer, this architecture relies entirely on managed, event-driven services (S3, SQS, Lambda, DynamoDB) to ingest, buffer, process, and record object metadata without managing servers.

Component	AWS Service	Architectural Purpose
Storage Ingestion	Amazon S3	Acts as the entry point for uploaded file objects and generates event notifications.
Message Buffer	Amazon SQS	Decouples S3 ingestion from execution, preventing downstream service throttling and enabling retries.
Compute Layer	AWS Lambda	Executes code statelessly on-demand to process incoming messages from SQS.
Database Tier	Amazon DynamoDB	Provides low-latency NoSQL storage to track and store image metadata records.

[S3 Bucket] ──► (s3:ObjectCreated:*) ──► [SQS Queue] ──► [AWS Lambda] ──► [

DynamoDB Table]

Description of the Project
In this lab, you will build an automated metadata processing pipeline. When a file is uploaded to Amazon S3, an S3 Event Notification is emitted directly to an Amazon Simple Queue Service (SQS) standard queue. An AWS Lambda function is configured to poll the SQS queue via an event source mapping, extract metadata (file size, name, timestamp), and write a record into an Amazon DynamoDB table.
This design demonstrates key SAA-C03 core architectural principles: **decoupling components** to build resilient systems, using **managed serverless technologies** to minimize operational overhead, and applying **least-privilege security access control** via IAM roles.

Step-by-Step Implementation Guide

Step 1: Building the Database & Ingestion Layer

- **Create S3 Storage Bucket:**
  - Bucket Name: image-upload-bucket-<unique-id>
  - Keep "Block all public access" enabled to enforce private storage.
- **Create DynamoDB Table:**
  - Table Name: ImageMetadata
  - Partition Key (PK): ImageId (Type: String)
  - Capacity Mode: On-Demand (to leverage free-tier pay-per-request pricing).

Step 2: Routing Logic & Decoupling with SQS

- **Provision SQS Queue:**
  - Queue Name: image-processing-queue (Standard Queue).
  - Configure Access Policy to permit S3 to send messages (sqs:SendMessage) matching your specific S3 bucket ARN.
- **Configure S3 Event Notifications:**
  - Navigate to S3 Bucket Properties ➔ Event Notifications.
  - Event Type: All object create events (s3:ObjectCreated:*).
  - Destination: SQS Queue ➔ Select image-processing-queue.

Step 3: Security & IAM Execution Role

- Create an IAM Role for Lambda named LambdaImageProcessorRole.
- Attach AWS Managed Policy: AWSLambdaBasicExecutionRole (enables writing logs to CloudWatch).
- Attach Inline Policy for SQS & DynamoDB Least-Privilege Access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes"
      ],
      "Resource": "arn:aws:sqs:*:*:image-processing-queue"
    },
```

```
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/ImageMetadata"
    }
  ]
}
```

Step 4: Deploying the Serverless Compute Layer (AWS Lambda)

- Create a Python 3.11 Lambda Function named ProcessImageMetadata.
- Attach the newly created IAM execution role LambdaImageProcessorRole.
- Add SQS Trigger: Set trigger source to image-processing-queue. Batch size: 10.
- Deploy the function code to extract event payload details and write to DynamoDB:

```
import json
import os
import urllib.parse
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('ImageMetadata')

def lambda_handler(event, context):
    for record in event['Records']:
        body = json.loads(record['body'])

        # S3 test notifications do not contain 'Records'
        if 'Records' not in body:
            continue

        for s3_record in body['Records']:
            bucket_name = s3_record['s3']['bucket']['name']
            object_key = urllib.parse.unquote_plus(s3_record['s3']['object']['key'])
            object_size = s3_record['s3']['object']['size']
            event_time = s3_record['eventTime']

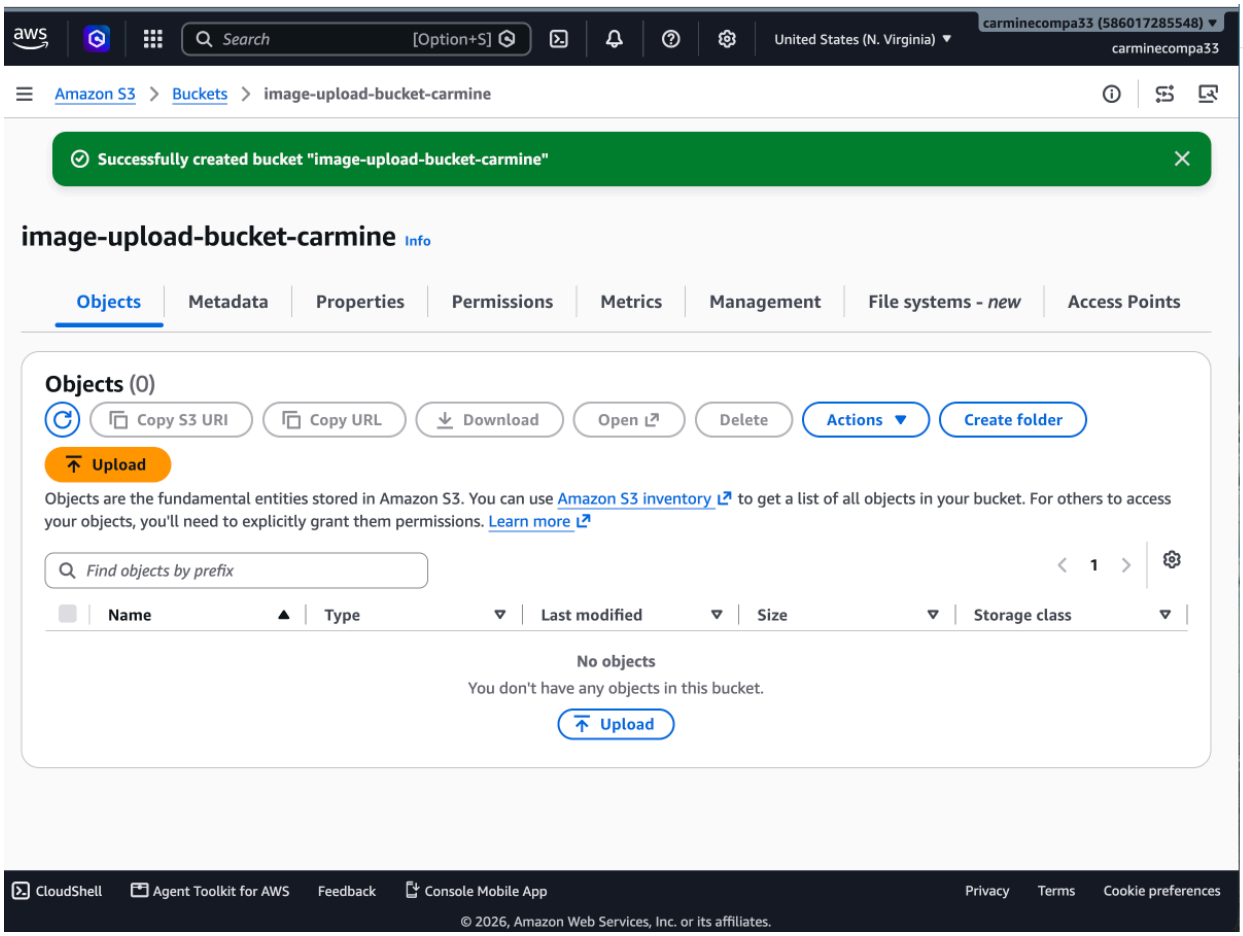
            # Insert metadata entry into DynamoDB
            table.put_item(
                Item={
                    'ImageId': object_key,
                    'Bucket': bucket_name,
                    'SizeBytes': object_size,
                    'ProcessedTimestamp': event_time
                }
            )
    return {'statusCode': 200, 'body': 'Success'}
```

Step 5: Testing & Verification

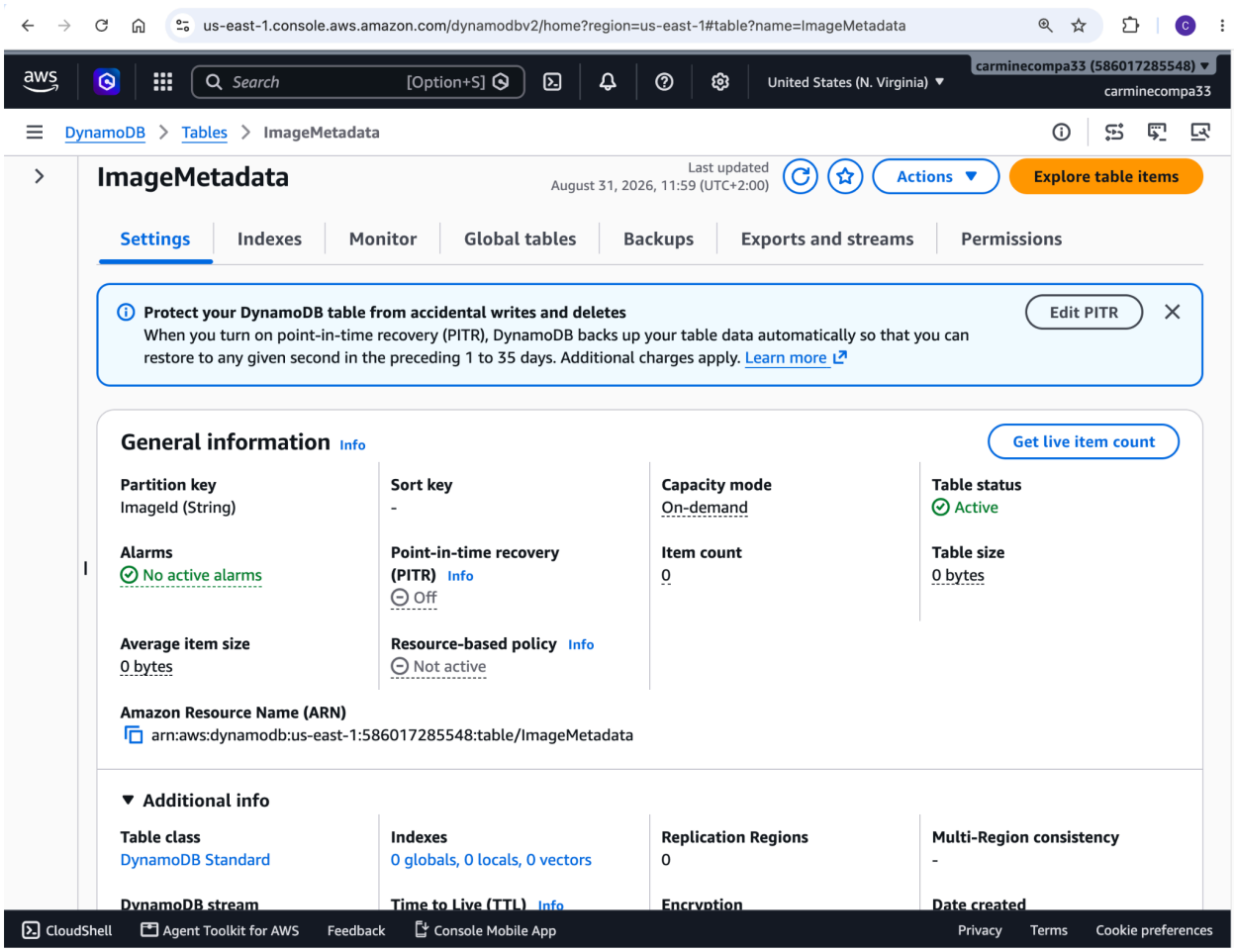
- **Upload Test File:** Upload an image or document (e.g., architecture-diagram.jpg) into your S3 bucket.
- **Verify Queue & Lambda Execution:** Check Amazon CloudWatch Logs under /aws/lambda/ProcessImageMetadata to inspect execution status and verify standard response status 200.
- **Database Inspection:** Open Amazon DynamoDB Console ➔ Explore Items ➔ Select ImageMetadata. Verify that the entry for architecture-diagram.jpg appears with correct file size and
- timestamp metadata.

Steps performed

I create s3 bucket

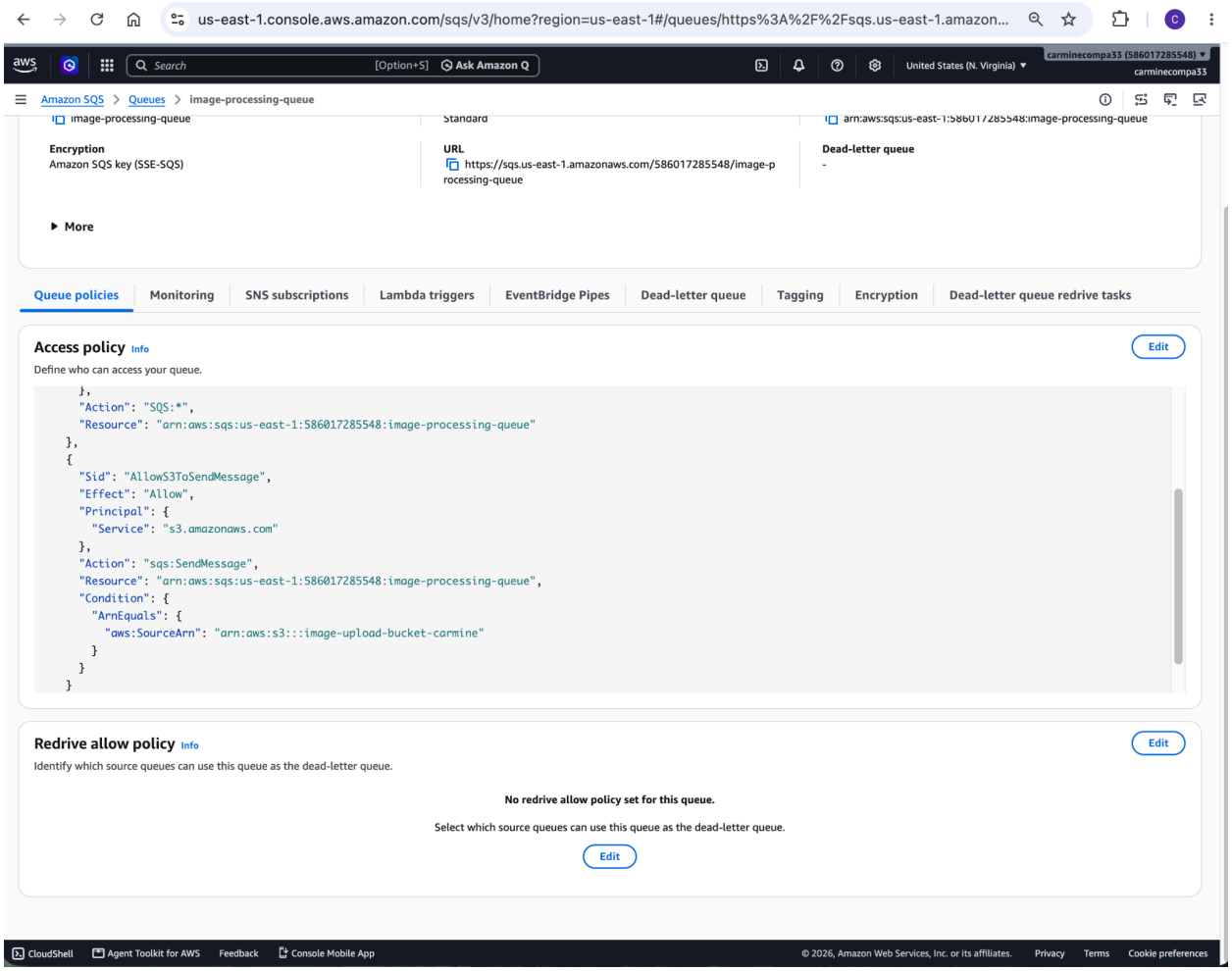


I create dynamodb

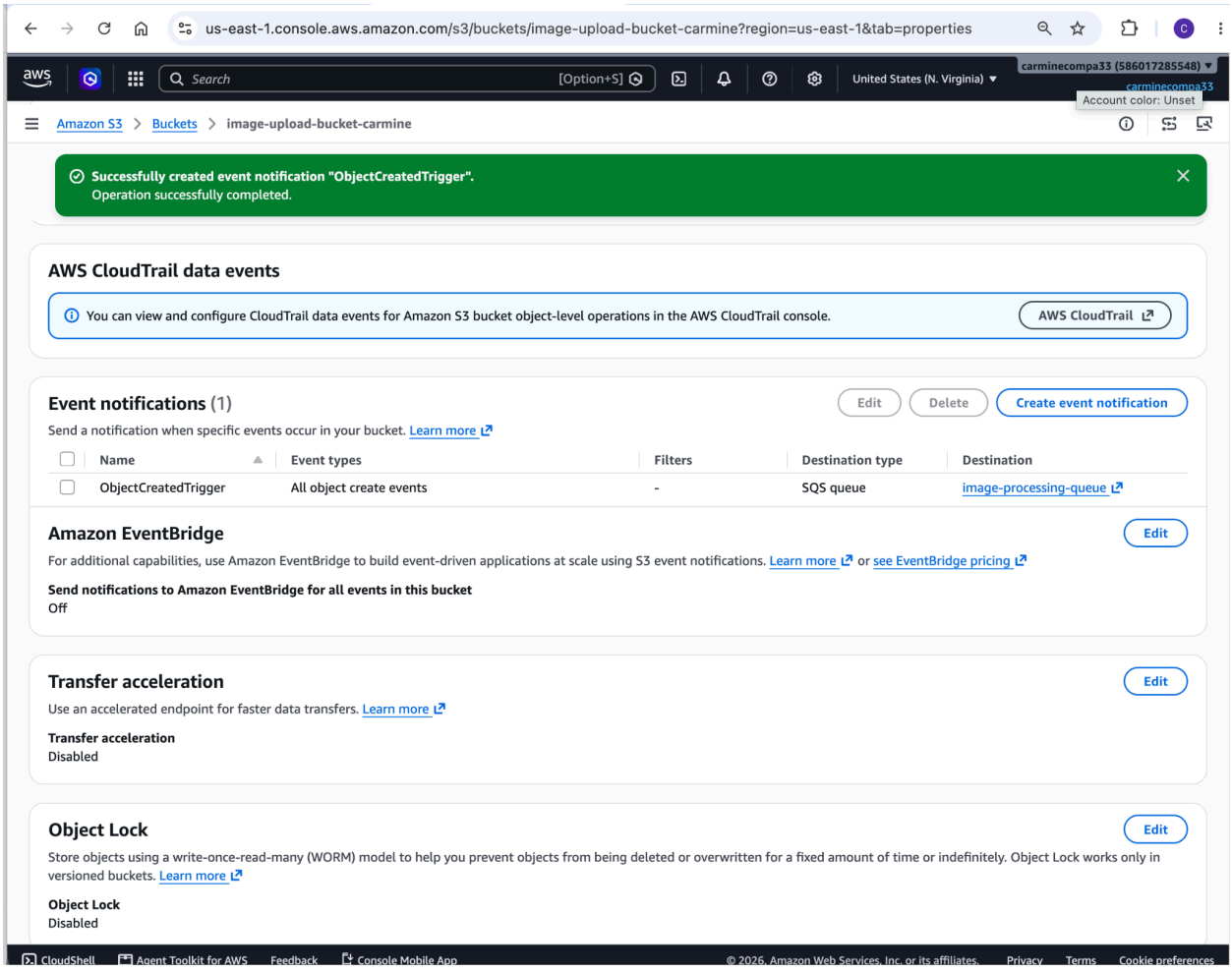


Created SQS queue

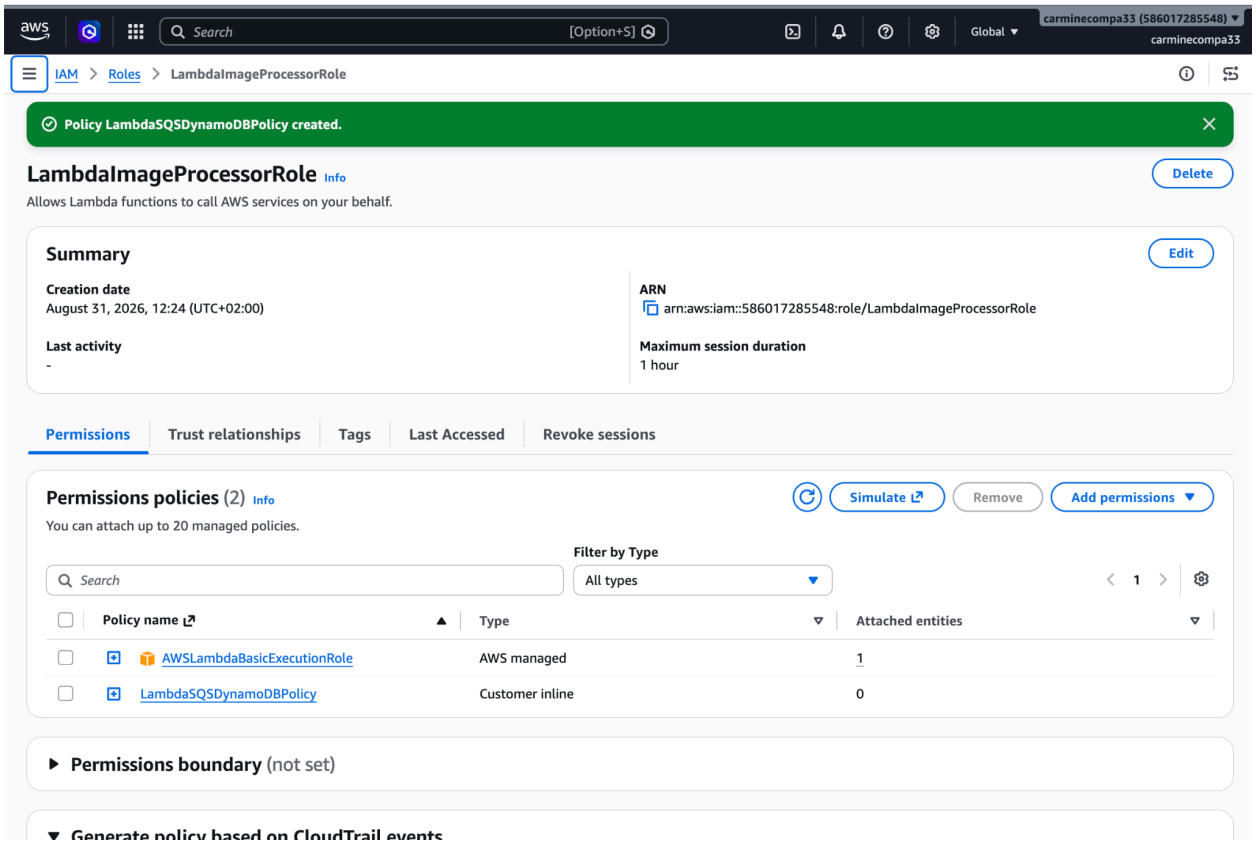
Allowed S3 bucket to drop notifications into SQS



Linked SQS to S3 bucket for create event notifications

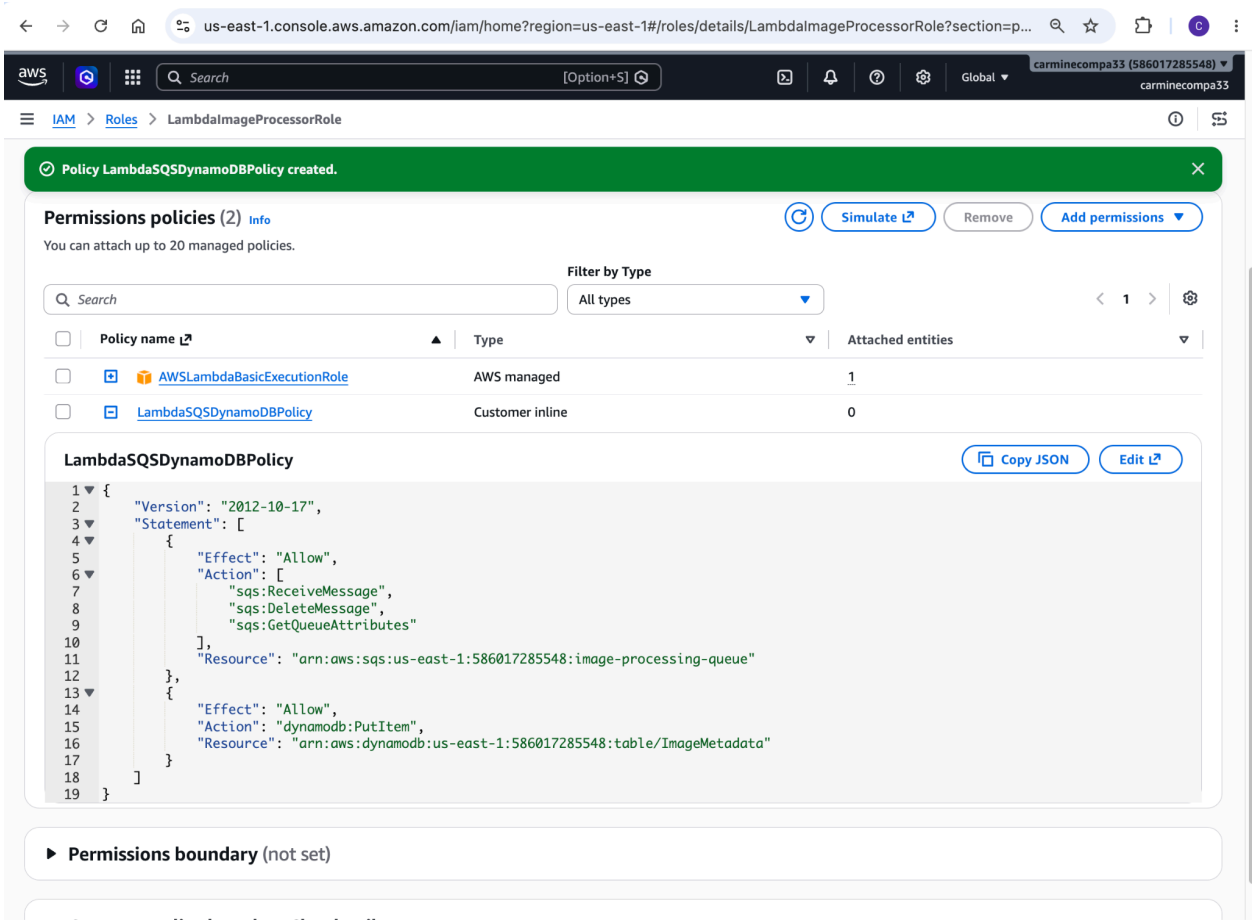


Create policy LambdaImageProcessorRole

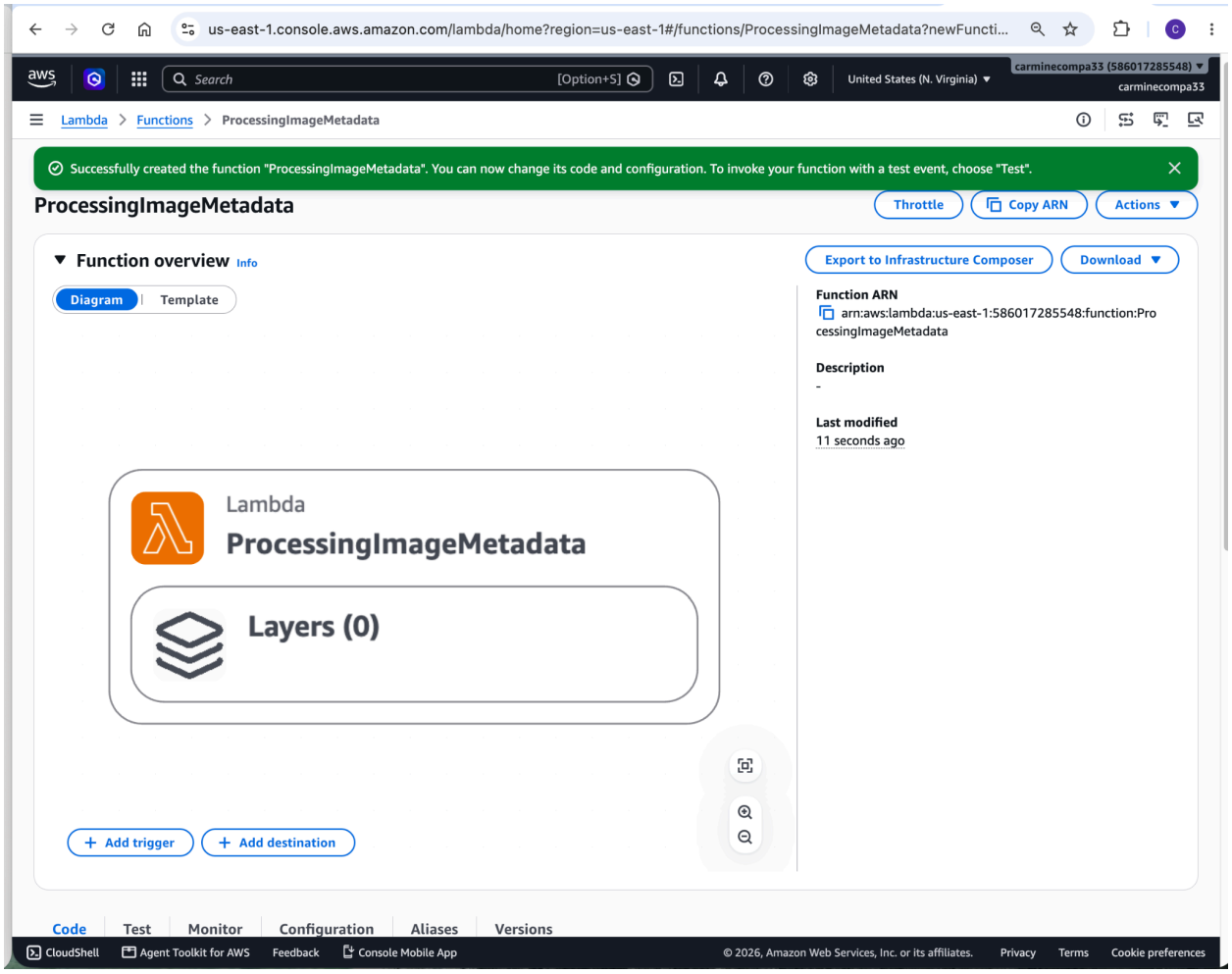


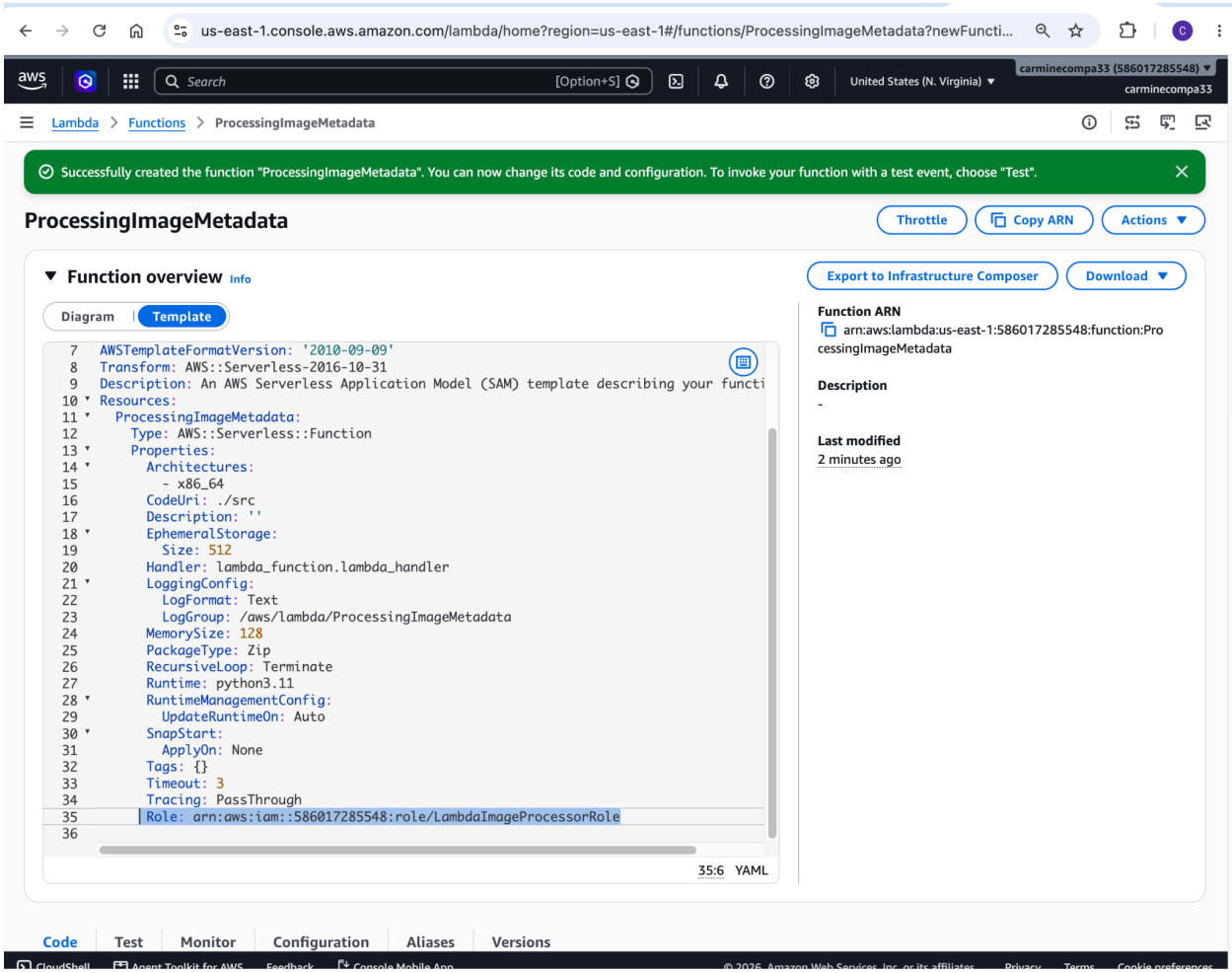
LambdaImageProcessorRole has sqs:ReceiveMessage, sqs:DeleteMessage and sqs:GetQueueAtttributes permissions

LambdaImageProcessorRole has dynamodb:PutItem permission

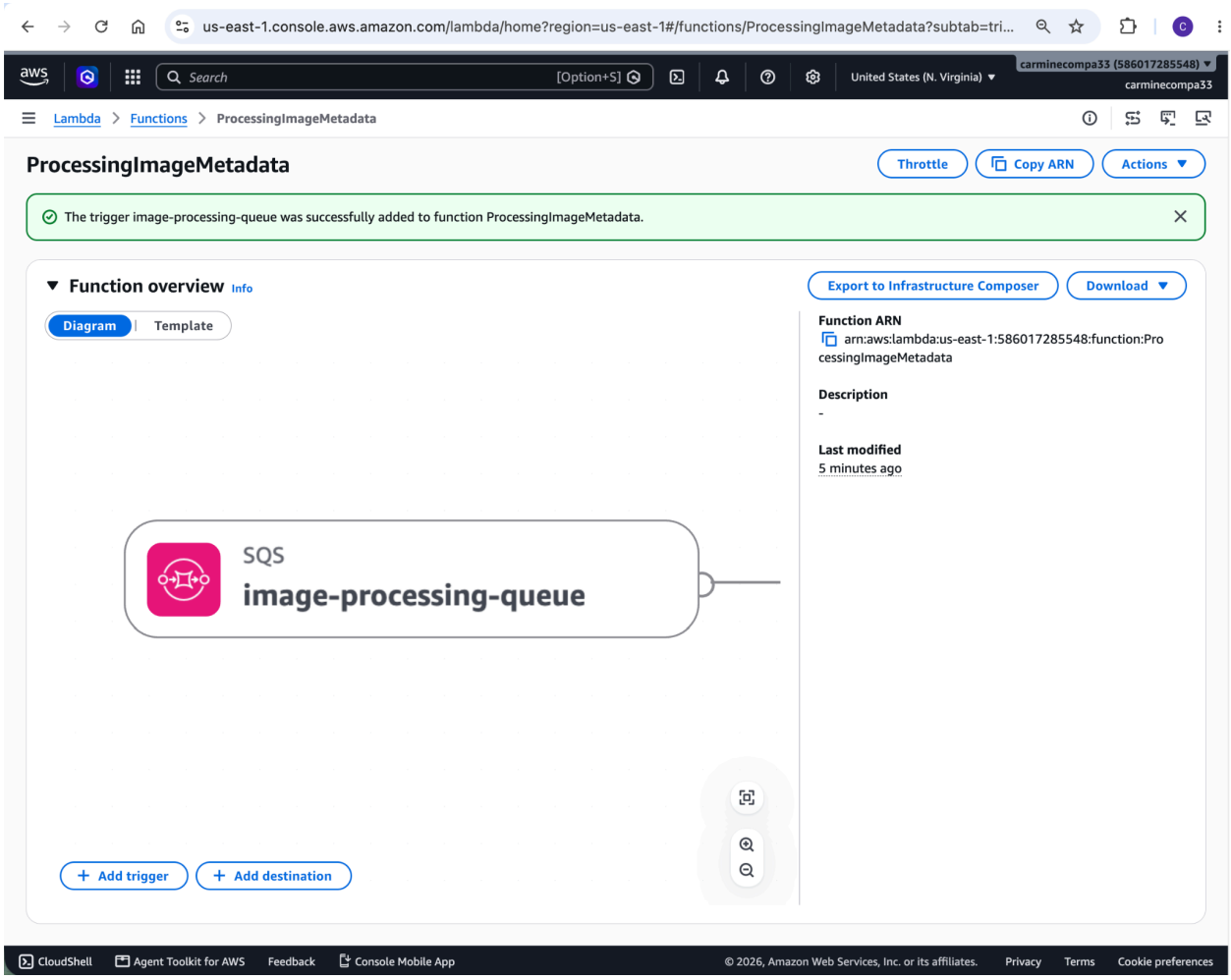


Created Lambda function with LambdaImageProcessorRole

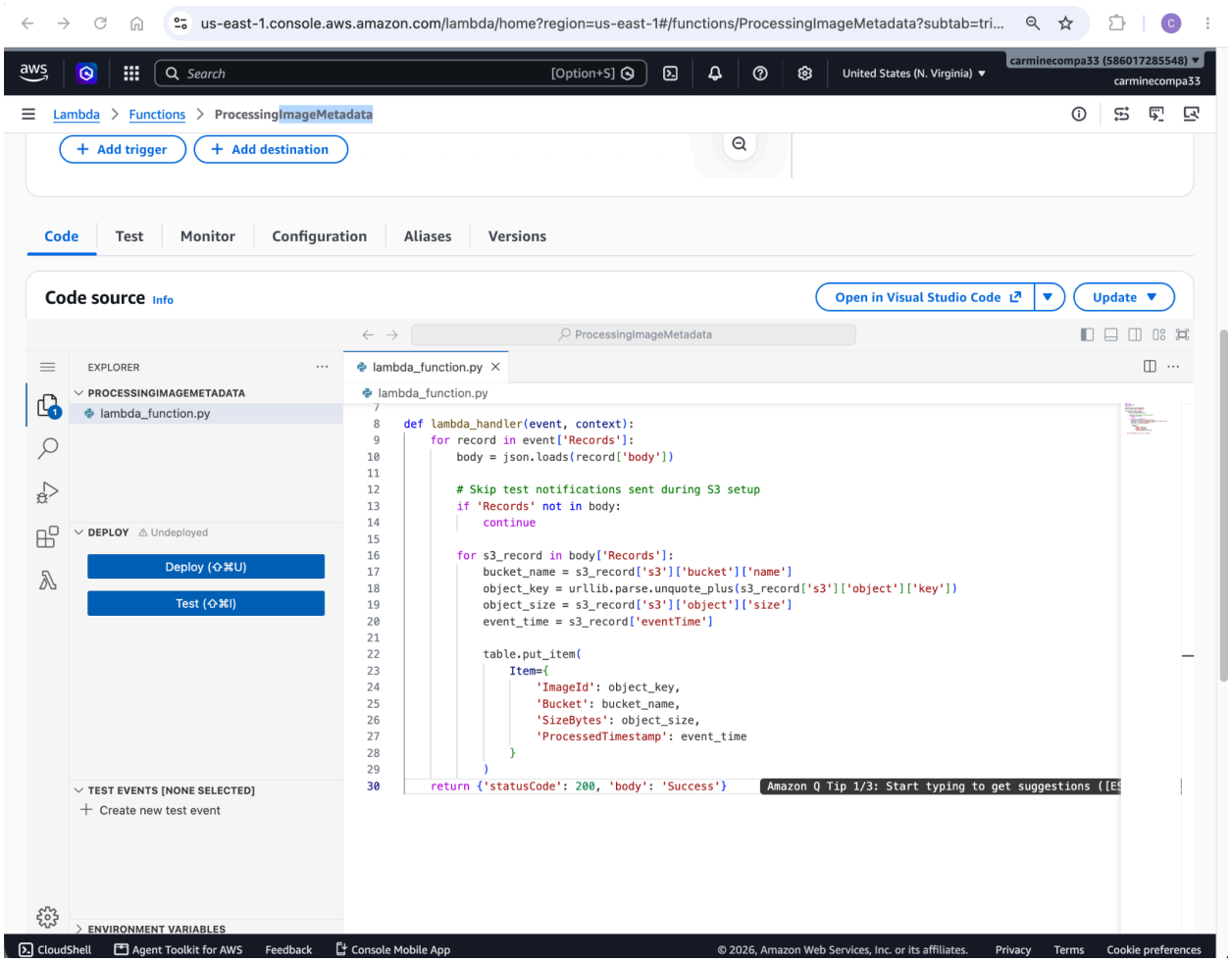

us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/ProcessingImageMetadata?newFuncti...
Search
[Option+S]
United States (N. Virginia)
carminecompa33 (586017285548)
carminecompa33
Lambda > Functions > ProcessingImageMetadata
Successfully created the function "ProcessingImageMetadata". You can now change its code and configuration. To invoke your function with a test event, choose "Test".
ProcessingImageMetadata
Throttle
Copy ARN
Actions
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
Function ARN
arn:aws:lambda:us-east-1:586017285548:function:ProcessingImageMetadata
Description
-
Last modified
11 seconds ago
Lambda
ProcessingImageMetadata
Layers (0)
Add trigger
Add destination
Code
Test
Monitor
Configuration
Aliases
Versions
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/ProcessingImageMetadata?newFuncti...
aws
Search
[Option+S]
United States (N. Virginia)
carminecompa33 (586017285548)
carminecompa33
Lambda > Functions > ProcessingImageMetadata
Successfully created the function "ProcessingImageMetadata". You can now change its code and configuration. To invoke your function with a test event, choose "Test".
ProcessingImageMetadata
Throttle
Copy ARN
Actions
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: An AWS Serverless Application Model (SAM) template describing your functi
Resources:
ProcessingImageMetadata:
Type: AWS::Serverless::Function
Properties:
Architectures:
- x86_64
CodeUri: ./src
Description: ''
EphemeralStorage:
Size: 512
Handler: lambda_function.lambda_handler
LoggingConfig:
LogFormat: Text
LogGroup: /aws/lambda/ProcessingImageMetadata
MemorySize: 128
PackageType: Zip
RecursiveLoop: Terminate
Runtime: python3.11
RuntimeManagementConfig:
UpdateRuntimeOn: Auto
SnapStart:
ApplyOn: None
Tags: {}
Timeout: 3
Tracing: PassThrough
Role: arn:aws:iam::586017285548:role/LambdaImageProcessorRole
35:6 YAML
Function ARN
arn:aws:lambda:us-east-1:586017285548:function:ProcessingImageMetadata
Description
-
Last modified
2 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions

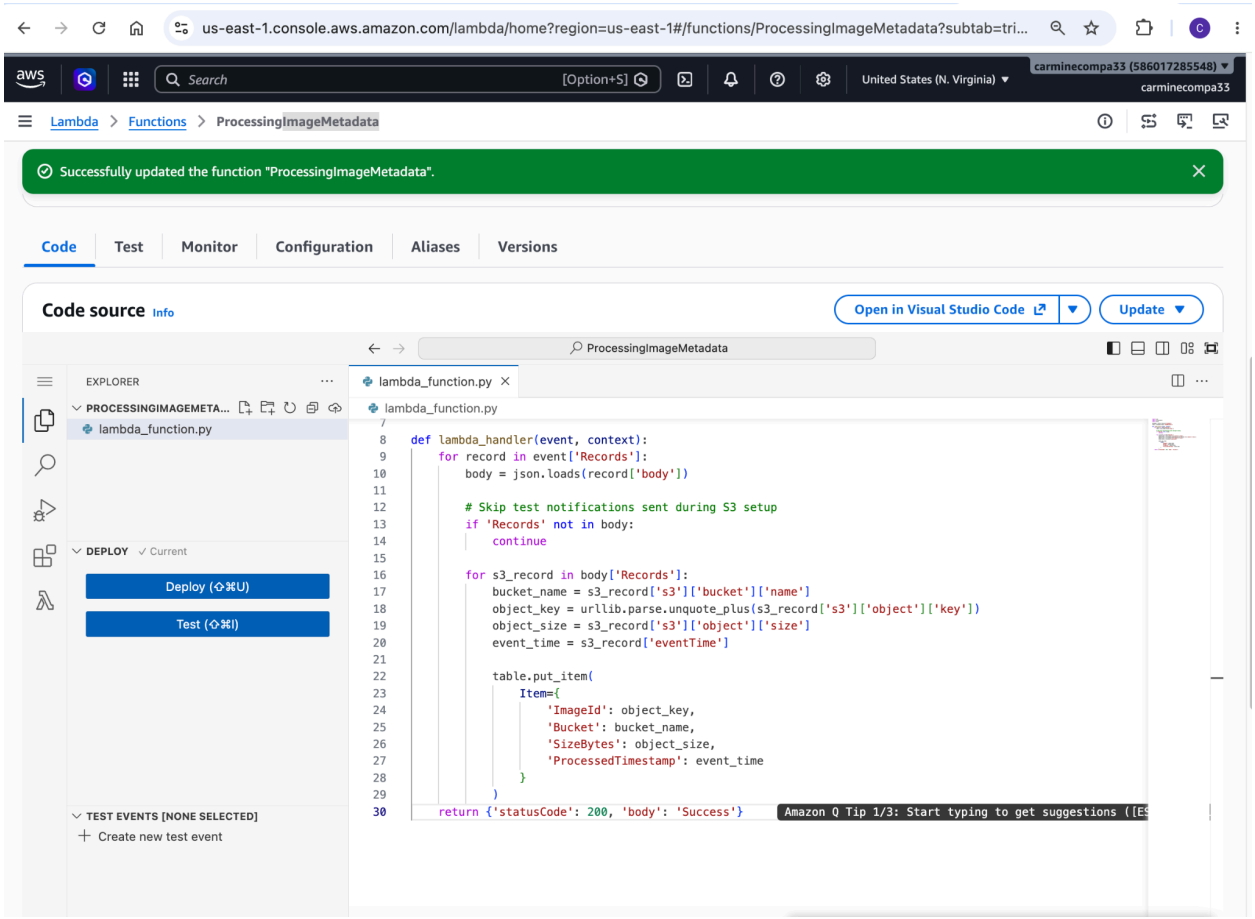
Added image-processing-queue as trigger for the Lambda function



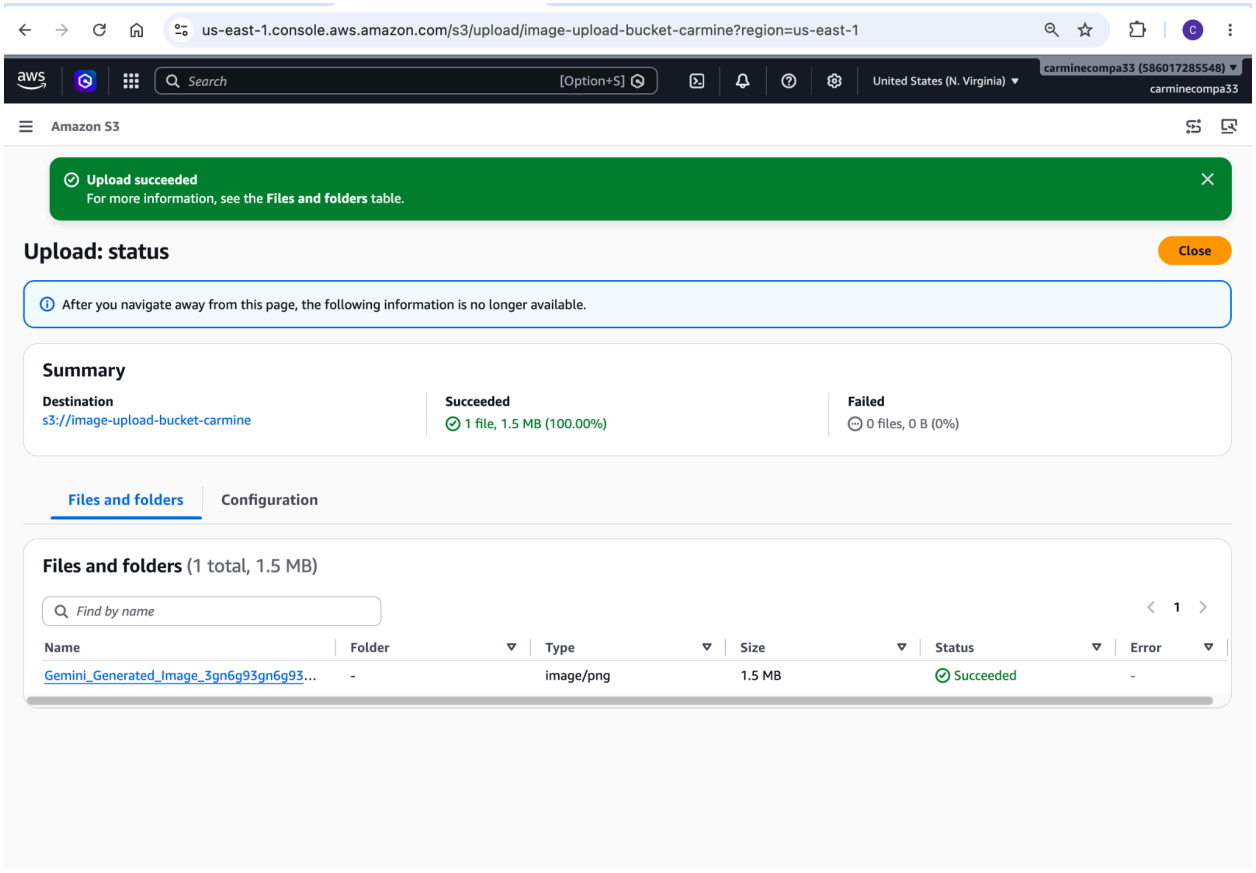
Added Lambda code



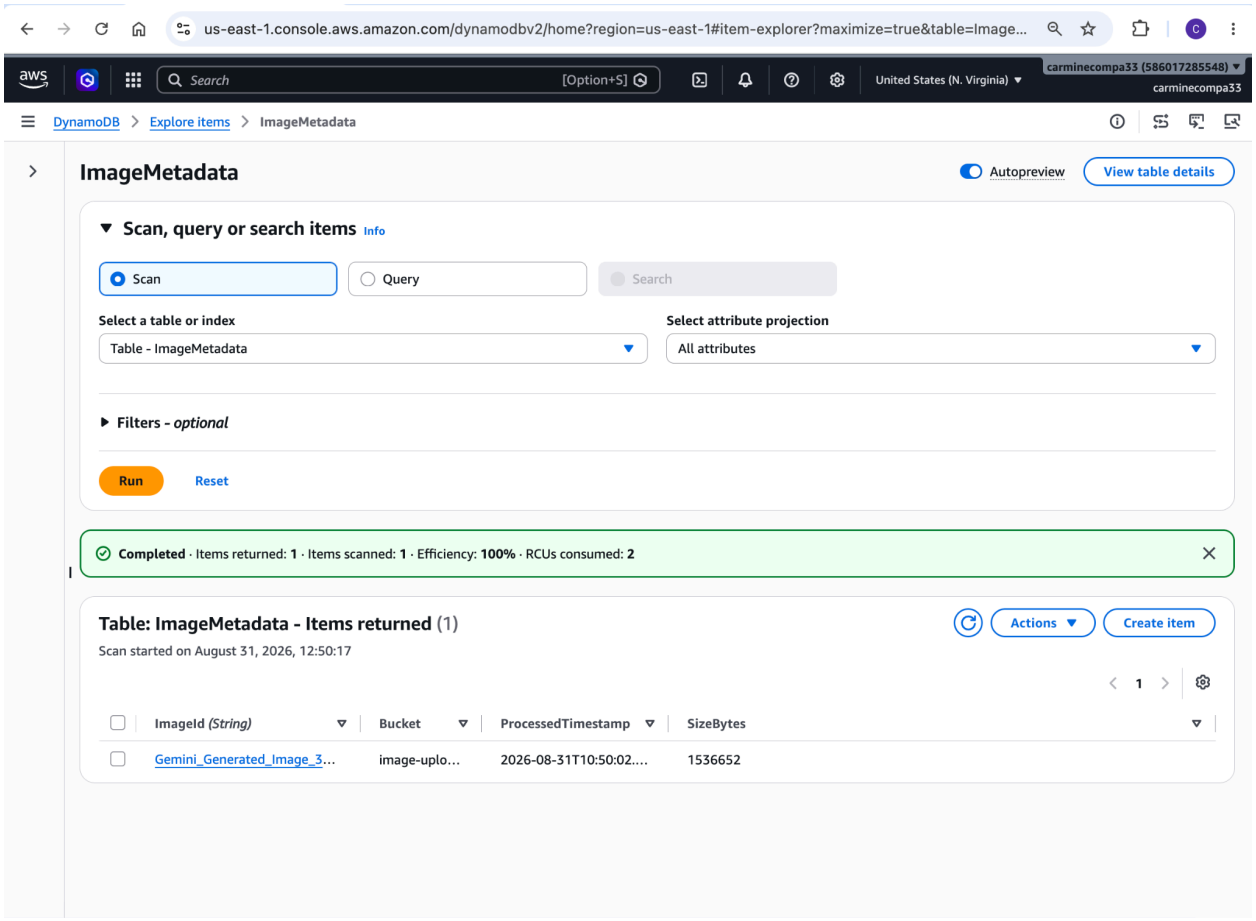
Deployed Lambda function



Uploaded an image to S3 bucket



Now I can see the file image in dynamo db!


us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#item-explorer?maximize=true&table=Image...
Search
[Option+S]
United States (N. Virginia)
carminecompa33 (586017285548)
carminecompa33
DynamoDB
Explore items
ImageMetadata
ImageMetadata
Autopreview
View table details
Scan, query or search items
Info
Scan
Query
Search
Select a table or index
Table - ImageMetadata
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2
Table: ImageMetadata - Items returned (1)
Scan started on August 31, 2026, 12:50:17
Actions
Create item
1
ImageId (String)
Bucket
ProcessedTimestamp
SizeBytes
Gemini_Generated_Image_3...
image-uplo...
2026-08-31T10:50:02....
1536652